SCILAB PRACTICAL

Exercises 1–6 | Questions, Programs & Expected Outputs

Course	B.Sc. Physics — Computation and Visualization with Scilab
Coverage	Matrix operations, linear algebra, series, factorial, prime numbers, plotting and Lissajous figures
Code style	Dark editor-style code blocks with syntax colours for comments, keywords, strings and functions

Prepared from the supplied Practical Exercise List 2026–27. Programs are written for Scilab; output values and plot appearance can vary slightly by version and environment.

How to use this booklet

Each exercise includes the question, a Scilab solution, and expected output or a description of the graph. Code blocks use an editor-inspired dark theme with coloured syntax. Copy each program into Scilab or save it as a .sce script before running.

Important: The source sheet's Exercise 1 wording contains a small inconsistency: its opening line mentions A as 3×4 and B as 4×3, while a later bullet says to create A as 3×3. To keep both sets of requested operations meaningful, Part 1(A) demonstrates the listed 3×3 column/row edits; Part 1(B) separately defines compatible 3×4 and 4×3 matrices for reshaping, addition and element-wise multiplication.

Contents

• Exercise 1(A) — Matrix editing operations

• Exercise 1(B) — Reshape, add, element-wise multiply, determinant and trace

• Exercise 2(A) — Solve three linear equations using AX = B

• Exercise 2(B) — Matrix inverse and timing comparison

• Exercise 3 — Sine series

• Exercise 4(A) — Factorial function

• Exercise 4(B) — Prime number test

• Exercise 5(A–D) — Function plots, polar plot, export and graph styling

• Exercise 6 — Lissajous figures

Exercise 1(A) — Matrix Editing Operations

Question

Create a 3×3 matrix A. Insert a column at the second-column position, delete the last row, insert a new row at the end, and replace every element greater than 10 with NaN.

Solution

```
clc;
clear;

// Step 1: Create a 3 x 3 matrix
A = [1 2 3;
4 11 6;
7 8 12];

disp("Original matrix A:");
disp(A);

// Step 2: Insert a new column in position 2
newCol = [10; 20; 30];
A = [A(:,1), newCol, A(:,2:$)];
disp("After inserting column 2:");
disp(A);

// Step 3: Delete the last row
A($,:) = [];
disp("After deleting the last row:");
disp(A);

// Step 4: Add a new row at the end
newRow = [13 14 15 16];
A = [A; newRow];
disp("After adding a new row:");
disp(A);

// Step 5: Replace values greater than 10 with NaN
A(A > 10) = %nan;
disp("Final matrix A:");
disp(A);
```

Expected output (key stages)

```
Original matrix A:
   1.    2.    3.
   4.   11.    6.
   7.    8.   12.

After inserting column 2:
   1.   10.    2.    3.
   4.   20.   11.    6.
   7.   30.    8.   12.

After deleting the last row:
   1.   10.    2.    3.
   4.   20.   11.    6.

After adding a new row:
   1.   10.    2.    3.
   4.   20.   11.    6.
```

```
   13.   14.   15.   16.

 Final matrix A:
    1.    10.     2.     3.
    4.    Nan.    Nan.   6.
    Nan.  Nan.    Nan.   Nan.
```

Note: the inserted row has four entries because the matrix has four columns after the column insertion.

Exercise 1(B) — Reshaping and Matrix Calculations

Question

Reshape A in possible dimensions; reshape A and B to compatible dimensions for addition and element-wise multiplication; extract a square matrix from A and calculate its determinant and trace.

Solution

```
clc;
clear;

// Define matrices with the dimensions stated in the exercise
A = [1 2 3 4;
5 6 7 8;
9 10 11 12];

B = [1 2 3;
4 5 6;
7 8 9;
10 11 12];

disp("A reshaped to 1 x 12:");
disp(matrix(A, 1, 12));

disp("A reshaped to 2 x 6:");
disp(matrix(A, 2, 6));

disp("A reshaped to 4 x 3:");
disp(matrix(A, 4, 3));

disp("A reshaped to 6 x 2:");
disp(matrix(A, 6, 2));

disp("A reshaped to 12 x 1:");
disp(matrix(A, 12, 1));

// Reshape B to 3 x 4 so it matches A
B1 = matrix(B, 3, 4);
disp("B reshaped to 3 x 4:");
disp(B1);

// Addition and element-wise multiplication
disp("A + B1:");
disp(A + B1);

disp("A .* B1:");
disp(A .* B1);

// Extract a 3 x 3 square matrix from A
S = A(:, 1:3);
disp("Square matrix S:");
disp(S);
disp("Determinant of S:");
```

```
disp(det(S));
disp("Trace of S:");
disp(trace(S));
```

Expected output

```
A reshaped to 1 x 12:
   1.    5.    9.    2.    6.   10.    3.    7.   11.    4.    8.   12.

A reshaped to 2 x 6:
   1.    9.    6.    3.   11.    8.
   5.    2.   10.    7.    4.   12.

B reshaped to 3 x 4:
   1.    4.    7.   10.
   2.    5.    8.   11.
   3.    6.    9.   12.

A + B1:
   2.    6.   10.   14.
   7.   11.   15.   19.
  12.   16.   20.   24.

A .* B1:
   1.    8.   21.   40.
  10.   30.   56.   88.
  27.   60.   99.  144.

Square matrix S:
   1.    2.    3.
   5.    6.    7.
   9.   10.   11.

Determinant of S:
   0.

Trace of S:
   18.
```

Explanation: Scilab's matrix() reshapes data in column-major order. The number of elements must remain 12. A 3×3 square matrix is taken from the first three columns of A; its determinant is 0 and its trace is 1 + 6 + 11 = 18.

Exercise 2(A) — Solving Linear Equations

Question

Write three linear equations in three unknowns and solve them using the matrix method AX = B.

Solution

```
clc;
clear;

// Equations:
// 2x + y - z = 8
// -3x - y + 2z = -11
// -2x + y + 2z = -3

A = [2 1 -1;
-3 -1 2;
-2 1 2];

B = [8; -11; -3];

// Solve AX = B using left division
X = A \ B;

disp("Solution [x; y; z]:");
disp(X);
```

Expected output

```
 Solution [x; y; z]:
    2.
    3.
   -1.
```

Therefore, x = 2, y = 3 and z = −1. The backslash operator solves the linear system without explicitly calculating the inverse.

Exercise 2(B) — Matrix Inverse and Timing

Question

Create a 3×3 matrix A. Find its inverse using inv() and left division (\), and compare the execution times using tic() and toc().

Solution

```
clc;
clear;

A = [2 1 1;
1 3 2;
1 0 2];

// Method 1: Explicit inverse
tic();
B = inv(A);
t1 = toc();

disp("Inverse using inv():");
disp(B);
disp("Time using inv():");
disp(t1);

// Method 2: Solve A*C = I
tic();
C = A \ eye(3, 3);
t2 = toc();

disp("Inverse using left division:");
disp(C);
disp("Time using left division:");
disp(t2);

// Compare measured times
if t1 < t2 then
disp("inv() was faster in this run.");
elseif t2 < t1 then
disp("Left division was faster in this run.");
else
disp("Both methods took equal time.");
end
```

Expected output

```
Inverse using inv():
   1.2   -0.4   -0.2
  -0.0    0.6   -0.6
  -0.6    0.2    0.8

Inverse using left division:
   1.2   -0.4   -0.2
  -0.0    0.6   -0.6
  -0.6    0.2    0.8

Time using inv():
(a small measured time)
```

```
Time using left division:
(a small measured time)

(The faster-method message depends on the actual run.)
```

The two inverse matrices should agree, apart from possible rounding. Timing results are machine- and run-dependent; for a small matrix, the difference may be negligible.

Exercise 3 — Sine Series

Question

Create a script named sineseries.sci to calculate sin(x) using n terms of its Taylor series. Take x and n as user inputs.

Series

$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$

Solution

```
clc;
clear;

x = input("Enter x in radians: ");
n = input("Enter number of terms: ");

s = 0;
term = x;

for k = 1:n
s = s + term;
// Generate the next term from the current term
term = -term * x^2 / ((2*k) * (2*k + 1));
end

disp("sin(x) using the series:");
disp(s);

disp("Scilab sin(x) for comparison:");
disp(sin(x));
```

Expected output (example: x = 1, n = 5)

```
Enter x in radians: 1
Enter number of terms: 5

sin(x) using the series:
   0.8414710

Scilab sin(x) for comparison:
   0.8414710
```

Save the script as sineseries.sci. Run it from the Scilab console with exec("sineseries.sci", -1) or use the editor's Run command. The angle x must be in radians.

Exercise 4(A) — Factorial Function

Question

Write a function to calculate the factorial of a given non-negative integer.

Solution

```
clc;
clear;

function f = factorial_num(n)
f = 1;
for i = 1:n
f = f * i;
end
endfunction

n = input("Enter a non-negative integer: ");

if n < 0 | n <> floor(n) then
disp("Invalid input: enter a non-negative integer.");
else
f = factorial_num(n);
disp("Factorial is:");
disp(f);
end
```

Expected output (n = 5)

```
Enter a non-negative integer: 5
Factorial is:
   120.
```

Remember: 0! = 1. This simple loop is intended for ordinary practical values; very large factorials can exceed numeric limits.

Exercise 4(B) — Prime Number Test

Question

Write a program to determine whether a given integer is prime or not.

Solution

```
clc;
clear;

n = input("Enter an integer: ");

if n <= 1 | n <> floor(n) then
disp("The number is not prime.");
else
isPrime = %t;

for i = 2:floor(sqrt(n))
if modulo(n, i) == 0 then
isPrime = %f;
break;
end
end

if isPrime then
disp("The number is prime.");
else
disp("The number is not prime.");
end
end
```

Expected output 1

```
Enter an integer: 7
The number is prime.
```

Expected output 2

```
Enter an integer: 12
The number is not prime.
```

A prime number is an integer greater than 1 with exactly two positive divisors: 1 and itself. Checking divisors only up to √n is sufficient.

Exercise 5(A) — Sine, Cosine and Tangent Plots

Question

Plot sine, cosine and tangent with titles, axis labels and axes through the origin. Arrange all three graphs vertically in one figure.

Solution

```
clc;
clear;

x = -%pi:0.01:%pi;
scf(0);

// Sine plot
subplot(3, 1, 1);
plot(x, sin(x), "r");
xtitle("Sine Function", "x (radians)", "sin(x)");
a = gca();
a.x_location = "origin";
a.y_location = "origin";

// Cosine plot
subplot(3, 1, 2);
plot(x, cos(x), "g");
xtitle("Cosine Function", "x (radians)", "cos(x)");
a = gca();
a.x_location = "origin";
a.y_location = "origin";

// Avoid tangent's vertical asymptotes at +/- pi/2
xt = (-%pi/2 + 0.05):0.01:(%pi/2 - 0.05);
subplot(3, 1, 3);
plot(xt, tan(xt), "b");
xtitle("Tangent Function", "x (radians)", "tan(x)");
a = gca();
a.x_location = "origin";
a.y_location = "origin";
```

Expected output

One figure with three vertically stacked plots: sin(x), cos(x), and tan(x). The tangent graph is restricted to the interval between −π/2 and π/2 to avoid plotting at its vertical asymptotes.

Exercise 5(B) — Polar Plot

Question

Plot r = exp(−θ/10) for θ from 0 to 10π using 200 equal intervals.

Solution

```
clc;
clear;

// 200 equal intervals means 201 sample points
theta = linspace(0, 10*%pi, 201);
r = exp(-theta/10);

scf(1);
polarplot(theta, r);
xtitle("Polar Plot: r = exp(-theta/10)");
```

Expected output

An inward spiral beginning at r = 1 and winding toward the origin. At θ = 10π, r = exp(−π) ≈ 0.0432.

Exercise 5(C) — Save Plots in Image Formats

Question

Save the plots in different image formats for later use.

Solution

Run these commands after the relevant figures have been created. Figure 0 is the three-function plot and figure 1 is the polar plot.

```
// Export figure 0: sine, cosine and tangent
xs2png(0, "functions.png");
xs2jpg(0, "functions.jpg");
xs2eps(0, "functions.eps");

// Export figure 1: polar plot
xs2png(1, "polarplot.png");
xs2jpg(1, "polarplot.jpg");
xs2eps(1, "polarplot.eps");
```

Expected result

Six files are created in Scilab's current working directory: functions.png, functions.jpg, functions.eps, polarplot.png, polarplot.jpg and polarplot.eps. Export function availability may vary by Scilab version.

Exercise 5(D) — Edit Graph Colours

Question

Reuse a graph and change its background colour, line colour and marker colour.

Solution

```
clc;
clear;

x = 0:0.2:10;
y = sin(x);

scf(2);
plot(x, y, "ro-");
xtitle("Edited Sine Graph", "x", "sin(x)");

// Change figure and axes background
f = gcf();
f.background = color("lightgrey");

a = gca();
a.background = color("lightgrey");

// Change the plotted curve's line and markers
curve = gce();
curve.children(1).foreground = color("purple");
curve.children(1).mark_mode = "on";
curve.children(1).mark_style = 9;
curve.children(1).mark_foreground = color("green");
curve.children(1).mark_background = color("green");
```

Expected output

A sine curve displayed with a light-grey background, purple line and green markers. Graphics object properties can differ between Scilab versions; if a property is unsupported, use the property editor for the selected polyline.

Exercise 6 — Lissajous Figures

Question

Plot Lissajous figures for frequency ratios 1:1, 1:2 and 3:2, with phase differences 0°, 45°, 90°, 135°, 180°, 225°, 270° and 315°. Label each subplot.

Theory

x = sin(a·t + δ), y = sin(b·t), where a:b is the frequency ratio and δ is the phase difference in radians.

Solution

```
clc;
clear;

t = linspace(0, 2*%pi, 1000);
phase = [0 45 90 135 180 225 270 315];
ratios = [1 1;
1 2;
3 2];

for k = 1:3
a = ratios(k, 1);
b = ratios(k, 2);
scf(k);

for j = 1:8
delta = phase(j) * %pi / 180;
x = sin(a*t + delta);
y = sin(b*t);

subplot(2, 4, j);
plot(x, y);
xtitle(msprintf("%d:%d, %d deg", a, b, phase(j)), "x", "y");
gca().isoview = "on";
end
end
```

Expected output

Three separate figures are created. Each figure contains eight subplots (2 rows × 4 columns), one for each phase difference. The curve shape changes with both the frequency ratio and phase difference.

Typical patterns

• Ratio 1:1: straight lines at phase differences 0° and 180°; circular/elliptical shapes at other phases (a circle at 90° or 270°).

• Ratio 1:2: two-frequency Lissajous patterns whose orientation and loops vary with phase.

• Ratio 3:2: more complex multi-loop patterns, also dependent on phase.

Practical tip: phase is supplied in radians to sin(), so convert degrees using phase × π/180. The isoview setting keeps equal scale on both axes.

Quick Revision: Scilab Commands

Command	Purpose
matrix(A, m, n)	Reshape a matrix to m rows and n columns (same element count).
A(:,1), A($,:)	Select a column; select the last row.
A \ B	Solve a linear system A·X = B.
inv(A), det(A), trace(A)	Inverse, determinant and trace.
tic(), toc()	Measure elapsed execution time.
modulo(n, i)	Remainder when n is divided by i.
subplot(m,n,k)	Select a panel in a grid of plots.
linspace(a,b,N)	Generate N evenly spaced points from a to b.
%pi, %nan, %t, %f	Scilab constants: π, NaN, true and false.

Before submitting

1. Run each program in Scilab and check the output on your own computer.
2. Save scripts with clear names, such as exercise1a.sce and sineseries.sci.
3. For plotting exercises, save or attach the generated figures if your instructor requires printouts.
4. The expected numerical formatting and timing values may differ slightly by Scilab version and system.